

CI/CD — TESTS, ARTIFACT PROMOTION AND SAFE RELEASES

Lab Manual

Four labs on one supplied pipeline for the Bookshelf api — Cloud Shell and the console only, nothing to code.

Contents

How to use this manual.....	3
Lab 1 — The gates, in order, and what happens when one fails.....	4
Lab 2 — Build once: digest, tag, scan, and 'latest' refused	7
Lab 3 — Promotion: staging automatically, prod behind an approval.....	9
Lab 4 — A bad release caught at 20 %, rolled back through the pipeline.....	12
Appendix — Environment, versions and the kit.....	15

How to use this manual

The slides teach the ideas and point to a lab; this manual has every step. The kit is a complete pipeline: a Cloud Build configuration with twelve steps, the two Bookshelf services, their tests, and a Cloud Deploy pipeline with staging and production targets. Nothing is coded. You run it, break it on purpose with one-line edits to an expected value or a setting, promote, approve, and roll back.

Conventions

- Cloud Shell blocks (teal) are copy-pasteable. Run them one block at a time, in order, from the kit folder `bookshelf-pipeline/`. Builds and rollouts take one to five minutes; the sheet has sleeps where a step must finish first.
- Expected panels show the real output format. Build ids, digests, revision names and timings differ; the shape is what matters.
- Console path lines give the equivalent clicks.
- Variables `PROJECT`, `R`, `AR`, `SHA`, `GOOD_SHA` and `DIGEST` are set in the blocks and reused across labs — if you open a new Cloud Shell tab, re-run the first block and Lab 2 step 1.
- There is no GitHub today. Builds are submitted from Cloud Shell against a local git history; the trigger and branch-protection slides show what a hosted repository adds.

Getting the kit into Cloud Shell

```
$ unzip -q gcp-cicd-labs.zip && cd bookshelf-pipeline && ls
README.md  cloudbuild.yaml  deploy  labs.sh  pyproject.toml  requirements-dev.txt
services  teardown.sh  tests
$ PROJECT=$(gcloud config get-value project); R=asia-south1; AR=$R-
docker.pkg.dev/$PROJECT/bookshelf
```

Starting state — what Session C left behind

Resource	Expected state	Check
bookshelf-api	runs as <code>bookshelf-api-min</code> , internal ingress, secret db-password mounted and pinned to 2	<code>gcloud run services describe bookshelf-api --region \$R --format='value(spec.template.spec.serviceAccountName)'</code>
bookshelf-deployer	roles/ <code>run.admin</code> on the project; <code>serviceAccountUser</code> on <code>bookshelf-api-min</code>	<code>gcloud iam service-accounts get-iam-policy bookshelf-api-min@\$PROJECT.iam.gserviceaccount.com</code>
bookshelf-web	public, VPC egress to reach the internal api	<code>curl -s \$(gcloud run services describe bookshelf-web --region \$R --format='value(status.url)') grep -c '' → 4</code>
Artifact Registry	repository <code>bookshelf</code> in <code>asia-south1</code>	<code>gcloud artifacts repositories describe bookshelf --location=\$R</code>

If any row is missing, the IAM session's `labs.sh` (Labs 1–3) recreates it in about 20 minutes; the Cloud Run session's kit recreates `bookshelf-web`.

Lab 1 — The gates, in order, and what happens when one fails

Estimated time	Difficulty	Environment	Kit
25 min	●●○	Cloud Shell, region asia-south1	gcp-cicd-labs.zip → bookshelf-pipeline/ (cloudbuild.yaml, services/, tests/, deploy/, labs.sh)

OBJECTIVE

The supplied pipeline run end to end with every gate passing and an image tagged by commit pushed and scanned; then a deliberately broken unit test stopping the build with no image produced; the step timings of both builds compared; and the contract test catching a renamed field before anything is built.

PREREQUISITES

- Starting state confirmed (previous page).
- Kit unzipped; you are in bookshelf-pipeline/.

Steps

Step 1 — Enable the APIs and give the kit a git history

Every build is named after a commit. The kit ships without one, so create it.

CLOUD SHELL

```
gcloud services enable cloudbuild.googleapis.com artifactregistry.googleapis.com
containerscanning.googleapis.com \
  ondemandscanning.googleapis.com clouddeploy.googleapis.com run.googleapis.com
git init -q -b main && git add -A && git -c user.name=grad -c
user.email=grad@example.com commit -q -m "bookshelf pipeline kit"
git log --oneline | head -1
```

EXPECTED

```
9f3c2e1 bookshelf pipeline kit
```

Step 2 — Read the pipeline — twelve steps, each a gate

CLOUD SHELL

```
grep -E "^ id:" cloudbuild.yaml
```

EXPECTED

```
- id: tooling
- id: lint
- id: typecheck
- id: unit-tests
- id: contract-tests
- id: dependency-scan
- id: build
- id: integration-start
- id: integration-check
- id: push
- id: vulnerability-scan
- id: digest
```

Open cloudbuild.yaml and read one step: name is the container it runs in, args is the command, waitFor is the ordering. The cheap gates come first; the three-minute build only starts after all of them pass.

Step 3 — Run it — every gate passes

Console path: Cloud Build → History → the running build → each step's log

CLOUD SHELL

```
SHA=$(git rev-parse --short HEAD); GOOD_SHA=$SHA
gcloud builds submit --config cloudbuild.yaml --substitutions=_SHA=$SHA .
```

EXPECTED

```
Creating temporary archive of 22 file(s) totalling 41.3 KiB before compression.
...
Step #1 - "lint": All checks passed!
Step #3 - "unit-tests": 7 passed in 0.31s
Step #4 - "contract-tests": 2 passed in 0.19s
Step #5 - "dependency-scan": No known vulnerabilities found
Step #8 - "integration-check": ok
Step #10 - "vulnerability-scan": CRITICAL findings: 0
Step #11 - "digest": sha256:ab12c3d4...
ID          CREATE_TIME      DURATION    SOURCE      IMAGES      STATUS
2b1824b6-... 2026-09-09T11:02:04 4m38s      gs://...    -           SUCCESS
```

Four to six minutes. Read cloudbuild.yaml while it runs. The first build also pulls python:3.13-slim onto the worker.

Step 4 — Break a unit test on purpose, commit, run again — it stops at unit-tests

Change one expected value: the test says the catalogue has 4 books; make it say 5.

CLOUD SHELL

```
sed -i 's/assert body\["books"\] == 4/assert body["books"] == 5/'
tests/unit/test_api.py
git commit -qam "break: wrong expected book count"
SHA=$(git rev-parse --short HEAD)
gcloud builds submit --config cloudbuild.yaml --substitutions=_SHA=$SHA . ; echo
"exit=$?"
gcloud artifacts docker images list $AR --include-tags --filter="tags:$SHA" --
format='value(tags)'
```

EXPECTED

```
Step #3 - "unit-tests": FAILED
tests/unit/test_api.py::test_index_reports_service_and_book_count - assert 4 == 5
Step #3 - "unit-tests": 1 failed, 6 passed in 0.33s
ERROR: build step 3 "python:3.13-slim" failed: step exited with non-zero status: 1
STATUS: FAILURE
exit=1
# the last command prints nothing: no image exists
for this commit
```

Steps 4–11 never ran. Nothing was built, nothing was pushed. That is the whole point of a gate.

Step 5 — Revert, and read the step timings of both builds**CLOUD SHELL**

```
git revert -q --no-edit HEAD
gcloud builds list --limit 2 --format='table(id,status,duration,substitutions._SHA) '
BUILD=$(gcloud builds list --limit 1 --format='value(id)')
gcloud builds describe $BUILD --
format='table(steps.id,steps.status,steps.timing.startTime.date("%H:%M:%S"),steps.timing.end
Time.date("%H:%M:%S"))'
```

EXPECTED

ID	STATUS	DURATION	_SHA
7e0c...	FAILURE	1m02s	e7d09a4
2b18...	SUCCESS	4m38s	9f3c2e1

ID	STATUS	START	END
tooling	SUCCESS	11:09:12	11:09:51
lint	SUCCESS	11:09:51	11:09:55

```

typecheck          SUCCESS  11:09:55  11:10:06
unit-tests         FAILURE  11:10:06  11:10:09
contract-tests     -        -        -

```

The failing build cost one minute; if the unit tests had run after the image build it would have cost four. Cheap gates first is a measurable saving, not a style preference.

Step 6 — The contract test — rename a field the web page reads

Rename the api's `title` field to `name` for one book, then run only the contract tests locally (the tools install into your Cloud Shell user site; nothing is committed).

CLOUD SHELL

```

sed -i 's/"title": "The Pragmatic Programmer"/"name": "The Pragmatic Programmer"/'
services/bookshelf-api/app.py
pip install --user -q -r requirements-dev.txt -r services/bookshelf-
api/requirements.txt -r services/bookshelf-web/requirements.txt
python3 -m pytest -q tests/contract 2>&1 | tail -3
git checkout -- services/bookshelf-api/app.py

```

EXPECTED

```

FAILED
tests/contract/test_contract.py::test_each_book_has_the_fields_the_web_page_reads -
AssertionError: missing {'title'}
1 failed, 1 passed in 0.21s

```

The api's own unit tests still pass — the api is internally consistent. The contract test is the one that protects the neighbour.

CHECK YOURSELF

- Build 1: SUCCESS, twelve steps, an image tagged with the commit.
- Build 2: FAILURE at unit-tests, no image for that commit.
- You can point at the timing table and say why lint runs before build.
- The contract test failed on a rename the unit tests did not notice.

Symptom	Cause	Fix
Build fails in seconds, empty log	compute default service account lacks Cloud Build roles	trainer's prep-project.sh (appendix); wait a minute
tooling step: 'No matching distribution'	a pin in requirements-dev.txt unavailable (network)	retry once; report to the trainer
push: permission denied	artifactregistry.writer missing on the compute SA	prep-project.sh
integration-check: connection refused	the container did not open port 8080 in time	retry; the step retries 10x over 20 s

Lab 2 — Build once: digest, tag, scan, and 'latest' refused

Estimated time	Difficulty	Environment	Kit
20 min	●●○	Cloud Shell, region asia-south1	gcp-cicd-labs.zip → bookshelf-pipeline/ (cloudbuild.yaml, services/, tests/, deploy/, labs.sh)

OBJECTIVE

The image from the passing build read by its commit tag and by its digest; the vulnerability scan re-run by hand and counted by severity; the effect of moving the gate from CRITICAL to HIGH measured; a 'latest' tag created, understood and deleted; the build provenance read.

PREREQUISITES

- Lab 1 complete (a passing build for GOOD_SHA).

Steps

Step 1 — Two names for one artifact — the commit tag and the digest

Console path: Artifact Registry → bookshelf → bookshelf-api → the image row shows both

CLOUD SHELL

```
SHA=${GOOD_SHA:-$(git rev-parse --short HEAD~2)}
gcloud artifacts docker images list $AR --include-tags --
format='table(package,tags,version,createTime)'
DIGEST=$(gcloud artifacts docker images describe $AR/bookshelf-api:$SHA --
format='value(image_summary.digest)')
echo "bookshelf-api@$DIGEST"
```

EXPECTED

PACKAGE	TAGS	VERSION
asia-south1-docker.pkg.dev/gcptraining-507817/bookshelf/bookshelf-api	1.0	
sha256:5c1e... 2026-09-08T...		
asia-south1-docker.pkg.dev/gcptraining-507817/bookshelf/bookshelf-api	1.1	
sha256:e4a7... 2026-09-09T...		
asia-south1-docker.pkg.dev/gcptraining-507817/bookshelf/bookshelf-api	9f3c2e1	
sha256:ab12c3... 2026-09-09T...		
bookshelf-api@sha256:ab12c3d4e5f6...		

Keep DIGEST — Lab 3 releases it. The 1.0 and 1.1 tags are from the earlier sessions.

Step 2 — The scan the pipeline ran, by hand

CLOUD SHELL

```
SCAN=$(gcloud artifacts docker images scan $AR/bookshelf-api:$SHA --remote --
format='value(response.scan)')
gcloud artifacts docker images list-vulnerabilities $SCAN --
format='value(vulnerability.effectiveSeverity)' | sort | uniq -c
```

EXPECTED

```
Scanning container image asia-south1-docker.pkg.dev/.../bookshelf-api:9f3c2e1
remotely...done.
      2 HIGH
     11 LOW
      6 MEDIUM
```

Numbers change week to week as CVEs are published against the Debian base. Zero CRITICAL is what the pipeline's gate checked.

Step 3 — What a stricter gate would have done**CLOUD SHELL**

```
N=$(gcloud artifacts docker images list-vulnerabilities $SCAN --
format='value(vulnerability.effectiveSeverity)' | grep -c '^HIGH$' || true); echo "HIGH
findings: $N"
gcloud artifacts docker images list-vulnerabilities $SCAN --
filter='vulnerability.effectiveSeverity=HIGH' --
format='table(vulnerability.shortDescription,vulnerability.packageIssue[0].affectedPackage,v
ulnerability.packageIssue[0].fixedVersion.name)'
```

EXPECTED

```
HIGH findings: 2
SHORT_DESCRIPTION  AFFECTED_PACKAGE  NAME
CVE-2026-1234      libc6               2.41-12
CVE-2026-2345      openssl            3.5.1-1
```

With `_SEVERITY_GATE: HIGH` in `cloudbuild.yaml` this build would have failed. The gate is a policy: pick it on purpose, and when it blocks, fix the base image or record an exception — never lower it quietly.

Step 4 — 'latest' — create it, see what it is, delete it**CLOUD SHELL**

```
gcloud artifacts docker tags add $AR/bookshelf-api:$SHA $AR/bookshelf-api:latest
gcloud artifacts docker images list $AR --include-tags --format='value(tags,version)'
| grep latest
gcloud artifacts docker tags delete $AR/bookshelf-api:latest --quiet
```

EXPECTED

```
9f3c2e1,latest  sha256:ab12c3...
```

Today latest means this digest. After the next push someone tags again and it means another. A deploy that says latest cannot be reproduced or rolled back to. Cloud Deploy (Lab 3) takes only a digest.

Step 5 — Provenance — who built this, from what**CLOUD SHELL**

```
gcloud artifacts docker images describe $AR/bookshelf-api:$SHA --show-provenance --
format='yaml(provenance_summary.provenance[0].build.intotoStatement.predicate.buildDefinitio
n.externalParameters)' 2>/dev/null | head -20
```

EXPECTED

```
provenance_summary:
  provenance:
    - build:
        intotoStatement:
          predicate:
            buildDefinition:
              externalParameters:
                buildConfigSource: { path: cloudbuild.yaml }
                substitutions: { _SHA: 9f3c2e1, ... }
```

If nothing prints, the build ran without SLSA provenance generation (it depends on project settings); the slide's example shows the shape. It is the record that ties the digest in production to a build id.

CHECK YOURSELF

- You can say the digest of the image you will promote.
- You know how many HIGH findings the base image carries today and what the gate would have done.
- latest no longer exists.

Lab 3 — Promotion: staging automatically, prod behind an approval

Estimated time	Difficulty	Environment	Kit
25 min	●●●	Cloud Shell, region asia-south1	gcp-cicd-labs.zip → bookshelf-pipeline/ (cloudbuild.yaml, services/, tests/, deploy/, labs.sh)

OBJECTIVE

The Cloud Deploy pipeline registered with two targets; a release of Lab 2's digest deployed automatically to a staging service; the same release promoted to prod, held for approval, approved, and its 20 % canary advanced to 100 %; and proof that staging and prod run the identical digest while prod keeps Session C's identity, secret and ingress.

PREREQUISITES

- Lab 2 complete — DIGEST and SHA set.
- bookshelf-deployer exists with actAs on bookshelf-api-min (Session C).

Steps

Step 1 — Roles for the pipeline identity; project id into the manifests

Console path: IAM & Admin → IAM → bookshelf-deployer → edit roles

CLOUD SHELL

```
gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:bookshelf-deployer@$PROJECT.iam.gserviceaccount.com --role=roles/clouddeploy.jobRunner
gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:bookshelf-deployer@$PROJECT.iam.gserviceaccount.com --role=roles/run.developer
sed -i "s/$PROJECT_ID/$PROJECT/g" deploy/clouddeploy.yaml deploy/run-service-staging.yaml deploy/run-service-prod.yaml
grep -n "$PROJECT" deploy/clouddeploy.yaml | head -3
```

EXPECTED

```
27: location: projects/gcptraining-507817/locations/asia-south1
30: serviceAccount: bookshelf-deployer@gcptraining-507817.iam.gserviceaccount.com
38: location: projects/gcptraining-507817/locations/asia-south1
```

jobRunner lets Cloud Deploy render and deploy as this identity; run.developer lets it create revisions; actAs on bookshelf-api-min it already has from Session C. Nothing at Editor level.

Step 2 — Register the pipeline and its two targets

Console path: Cloud Deploy → Delivery pipelines → bookshelf-api

CLOUD SHELL

```
gcloud deploy apply --file=deploy/clouddeploy.yaml --region=$R
gcloud deploy delivery-pipelines describe bookshelf-api --region=$R --format='value(Targets)'
```

EXPECTED

```
Waiting for the operation on resource projects/.../deliveryPipelines/bookshelf-api...done.
Created Cloud Deploy resource: projects/.../locations/asia-south1/targets/staging.
Created Cloud Deploy resource: projects/.../locations/asia-south1/targets/prod.
[{'Target': 'projects/.../targets/staging'}, {'Target': 'projects/.../targets/prod'}]
```

Step 3 — A release — the digest into both manifests; staging deploys on its own**CLOUD SHELL**

```
gcloud deploy releases create r-$SHA --delivery-pipeline=bookshelf-api --region=$R \
--source=deploy --images=bookshelf-api=$AR/bookshelf-api@$DIGEST
sleep 90; gcloud deploy rollouts list --release=r-$SHA --delivery-pipeline=bookshelf-
api --region=$R --format='table(name.basename(),targetId,state,approvalState)'
STG_URL=$(gcloud run services describe bookshelf-api-staging --region $R --
format='value(status.url)')
curl -s -H "Authorization: Bearer $(gcloud auth print-identity-token)" $STG_URL/ ;
echo
```

EXPECTED

```
Creating temporary archive of 4 file(s)... Created Cloud Deploy release r-9f3c2e1.
Creating rollout projects/.../rollouts/r-9f3c2e1-to-staging-0001 in target staging
NAME                                TARGET_ID  STATE      APPROVAL_STATE
r-9f3c2e1-to-staging-0001          staging     SUCCEEDED  DOES NOT NEED APPROVAL
{"books":4,"greeting":"Bookshelf catalogue (staging)","service":"bookshelf-
api","version":"staging"}
```

A new Cloud Run service bookshelf-api-staging now exists, deployed by bookshelf-deployer, running the digest. If the rollout is still IN_PROGRESS, wait 30 s and list again.

Step 4 — Promote to prod — and watch it wait for you

Console path: Cloud Deploy → bookshelf-api → prod rollout → Review

CLOUD SHELL

```
gcloud deploy releases promote --release=r-$SHA --delivery-pipeline=bookshelf-api --
region=$R --quiet
sleep 15; gcloud deploy rollouts list --release=r-$SHA --delivery-pipeline=bookshelf-
api --region=$R --format='table(name.basename(),targetId,state,approvalState)'
```

EXPECTED

NAME	TARGET_ID	STATE	APPROVAL_STATE
r-9f3c2e1-to-prod-0001	prod	PENDING APPROVAL	NEEDS APPROVAL
r-9f3c2e1-to-staging-0001	staging	SUCCEEDED	DOES_NOT_NEED_APPROVAL

Step 5 — Approve; the prod stage is a canary — advance it to 100 %

Console path: the rollout → Approve; later → Advance

CLOUD SHELL

```
ROLLOUT=$(gcloud deploy rollouts list --release=r-$SHA --delivery-pipeline=bookshelf-
api --region=$R --filter='targetId=prod' --format='value(name.basename())')
gcloud deploy rollouts approve $ROLLOUT --release=r-$SHA --delivery-
pipeline=bookshelf-api --region=$R --quiet
sleep 90; gcloud deploy rollouts describe $ROLLOUT --release=r-$SHA --delivery-
pipeline=bookshelf-api --region=$R --format='yaml(state,phases[].id,phases[].state)'
gcloud run services describe bookshelf-api --region $R --
format='value(status.traffic)'
gcloud deploy rollouts advance $ROLLOUT --release=r-$SHA --delivery-
pipeline=bookshelf-api --region=$R --quiet
sleep 60; gcloud run services describe bookshelf-api --region $R --
format='value(status.traffic)'
```

EXPECTED

```
state: IN_PROGRESS
phases:
- {id: canary-20, state: SUCCEEDED}
- {id: stable, state: PENDING}
{'percent': 80, 'revisionName': 'bookshelf-api-00012-abc'};{'percent': 20,
'revisionName': 'bookshelf-api-00013-def'}
Advanced rollout r-9f3c2e1-to-prod-0001 to phase stable.
{'percent': 100, 'revisionName': 'bookshelf-api-00013-def'}
```

You approved your own release because you are Owner of the sandbox. In production the approver role is a group that does not include the author.

Step 6 — The proof — same digest in both, prod unchanged in everything else

CLOUD SHELL

```
for S in bookshelf-api-staging bookshelf-api; do
  echo "== $S"; gcloud run services describe $S --region $R --
format='value(spec.template.spec.containers[0].image,spec.template.spec.serviceAccountName,m
etadata.annotations."run.googleapis.com/ingress")'
done
```

EXPECTED

```
== bookshelf-api-staging
asia-south1-docker.pkg.dev/.../bookshelf-api@sha256:ab12c3... bookshelf-api-min@... all
== bookshelf-api
asia-south1-docker.pkg.dev/.../bookshelf-api@sha256:ab12c3... bookshelf-api-min@...
internal
```

Identical digest; different ingress, greeting and scaling. The identity, the secret mount and the internal ingress from Session C are now written in the prod manifest — the pipeline owns them.

CHECK YOURSELF

- Two rollouts SUCCEEDED, one of them after your approval.
- Traffic went 80/20 then 100 on the new revision.
- Both services show the same @sha256 digest.

Symptom	Cause	Fix
releases create: 'image ... not found'	DIGEST empty	re-run Lab 2 step 1
rollout FAILED: 'actAs' / permission denied	bookshelf-deployer lost Session C's serviceAccountUser binding	gcloud iam service-accounts add-iam-policy-binding bookshelf-api-min@... --member=serviceAccount:bookshelf-deployer@... --role=roles/iam.serviceAccountUser
rollout FAILED: secret db-password	Session C's secret gone or version 2 disabled	re-run IAM Lab 2 steps 2–3 and 7
describe shows phases: none	target strategy not applied (apply ran before the edit)	gcloud deploy apply again; create a new release

Lab 4 — A bad release caught at 20 %, rolled back through the pipeline

Estimated time	Difficulty	Environment	Kit
25 min	●●●	Cloud Shell, region asia-south1	gcp-cicd-labs.zip → bookshelf-pipeline/ (cloudbuild.yaml, services/, tests/, deploy/, labs.sh)

OBJECTIVE

A release of the same digest with a faulty setting promoted to prod and approved; the canary holding 20 % of traffic; failures observed from outside through bookshelf-web and in the logs; the rollout NOT advanced; a rollback through Cloud Deploy restoring 100 % on the good revision; and a fix-forward release started the normal way.

PREREQUISITES

- Lab 3 complete.
- bookshelf-web deployed and reaching the api (starting-state table).

Steps

Step 1 — Baseline — twenty page loads, zero failures

CLOUD SHELL

```
WEB_URL=$(gcloud run services describe bookshelf-web --region $R --
format='value(status.url)')
for i in $(seq 1 20); do curl -s "$WEB_URL/" | grep -c "could not reach"; done | sort
| uniq -c
```

EXPECTED

20 0

Twenty loads, each with zero 'could not reach' lines. If every line says 1, bookshelf-web cannot reach the internal api — see troubleshooting.

Step 2 — The bad release — same digest, a setting that makes half the /books calls fail

FAULT_RATE is a setting bookshelf-api 1.2 reads at startup. Add it to the prod manifest (a two-line YAML edit made by a short text edit), commit, and release. Nothing about the image changes.

CLOUD SHELL

```
python3 - <<'EOF'
p='deploy/run-service-prod.yaml'; s=open(p).read()
s=s.replace('          - name: APP_VERSION\n          value: prod\n', '          - name:
APP_VERSION\n          value: prod\n          - name: FAULT_RATE\n          value:
"0.5"\n',1)
open(p,'w').write(s)
EOF
grep -n -A1 FAULT_RATE deploy/run-service-prod.yaml
git commit -qam "release: enable FAULT_RATE (deliberately bad)"
gcloud deploy releases create r-bad --delivery-pipeline=bookshelf-api --region=$R --
source=deploy --images=bookshelf-api=$R/bookshelf-api@$DIGEST
sleep 90; gcloud deploy releases promote --release=r-bad --delivery-
pipeline=bookshelf-api --region=$R --quiet
sleep 15; BAD=$(gcloud deploy rollouts list --release=r-bad --delivery-
pipeline=bookshelf-api --region=$R --filter='targetId=prod' --
format='value(name.basename())')
gcloud deploy rollouts approve $BAD --release=r-bad --delivery-pipeline=bookshelf-api
--region=$R --quiet
sleep 90; gcloud run services describe bookshelf-api --region $R --
format='value(status.traffic)'
```

EXPECTED

```
24:         - name: FAULT_RATE
25-         value: "0.5"
Created Cloud Deploy release r-bad.
{'percent': 80, 'revisionName': 'bookshelf-api-00013-def'};{'percent': 20,
'revisionName': 'bookshelf-api-00014-ghi'}
```

Staging got it first and 'passed' — nobody looked. That is what the canary is for.

Step 3 — Watch the canary from the outside — and in the logs

Console path: Cloud Run → bookshelf-api → Logs, filter 'simulated fault'

CLOUD SHELL

```
for i in $(seq 1 30); do curl -s "$WEB_URL/" | grep -c "could not reach"; done | sort
| uniq -c
gcloud run services logs read bookshelf-api --region $R --limit 40 | grep -c
"simulated fault"
```

EXPECTED

```
26 0
4 1
3
```

About one load in ten: 20 % of requests reach the canary, and half of those fail. Twenty-six good, four bad — the number varies. Do NOT advance.

Step 4 — Roll back through the pipeline

Console path: Cloud Deploy → bookshelf-api → prod → Rollback

CLOUD SHELL

```
gcloud deploy targets rollback prod --delivery-pipeline=bookshelf-api --region=$R --
quiet
sleep 90; gcloud deploy rollouts list --delivery-pipeline=bookshelf-api --region=$R --
filter='targetId=prod' --format='table(name.basename(),state,phases[0].id)' | head -5
gcloud run services describe bookshelf-api --region $R --
format='value(status.traffic)'
for i in $(seq 1 20); do curl -s "$WEB_URL/" | grep -c "could not reach"; done | sort
| uniq -c
```

EXPECTED

```
Rolling back target prod to release r-9f3c2e1...
NAME                                STATE      ID
r-9f3c2e1-to-prod-0002             SUCCEEDED stable
r-bad-to-prod-0001                 CANCELLED  canary-20
r-9f3c2e1-to-prod-0001             SUCCEEDED  canary-20
{'percent': 100, 'revisionName': 'bookshelf-api-00015-jkl'}
20 0
```

The rollback is a new rollout of the previous release — same identity, same manifests, same record. If it pauses in a canary phase, advance it: `gcloud deploy rollouts advance r-9f3c2e1-to-prod-0002 --release=r-9f3c2e1 ...` Rollback undoes the release; it does not undo a schema change or a secret rotation — those roll back separately.

Step 5 — Fix forward — remove the fault and release the normal way**CLOUD SHELL**

```
sed -i '/FAULT_RATE/,+1d' deploy/run-service-prod.yaml && grep -c FAULT_RATE
deploy/run-service-prod.yaml
git commit -qam "release: remove FAULT_RATE"
gcloud deploy releases create r-fixed --delivery-pipeline=bookshelf-api --region=$R --
source=deploy --images=bookshelf-api=$AR/bookshelf-api@$DIGEST
```

EXPECTED

0

Created Cloud Deploy release r-fixed.

Then promote → approve → advance, exactly as Lab 3 steps 4–5, if time allows. Leave everything running: the Observability and Incident sessions use this pipeline.

CHECK YOURSELF — AND LEAVE EVERYTHING RUNNING

- The bad release reached 20 % and you saw roughly one failure in ten.
- You rolled back with one command and the page went clean.
- prod is on 100 % of a good revision; the pipeline, both services and the staging service stay up.

Symptom	Cause	Fix
Baseline shows 20 1 (every load fails)	bookshelf-web cannot reach the internal api (no VPC egress)	gcloud run services update bookshelf-web --region \$R --network default --subnet default --vpc-egress private-ranges-only
Canary shows 0 failures in 30 loads	traffic split not applied yet, or the page is cached	check status.traffic shows 20 %; run 30 more loads
rollback rollout stuck IN_PROGRESS	paused at a canary phase	advance it (see step 4 note)

Appendix — Environment, versions and the kit

Versions verified 2026-09-09

Item	Value
gcloud	583.x (Cloud Shell)
Pipeline tools	pytest 9.0.3 · ruff 0.15.11 · mypy 1.20.2 · pip-audit 2.10.1 (requirements-dev.txt) — all four run clean on the kit
Cloud Build	cloudbuild.yaml, 12 steps; substitutions _SHA, _SEVERITY_GATE (CRITICAL); logging CLOUD_LOGGING_ONLY
Artifact Analysis	automatic scanning (containerscanning API) + on-demand (ondemandscanning API) in the pipeline
Cloud Deploy	DeliveryPipeline bookshelf-api; targets staging (auto) and prod (requireApproval, canary [20] with automaticTrafficControl); execution as bookshelf-deployer
Cloud Run	bookshelf-api (prod, internal) and bookshelf-api-staging; manifests in deploy/
bookshelf-api	1.2 — adds FAULT_RATE (default 0) for Lab 4

The kit

File	Purpose
cloudbuild.yaml	the pipeline
services/	bookshelf-api 1.2, bookshelf-web — supplied complete
tests/unit, tests/contract	the suites the gates run
deploy/clouddeploy.yaml, scaffold.yaml, run-service-*.yaml	Cloud Deploy pipeline, targets, manifests (PROJECT_ID filled in Lab 3)
labs.sh	Labs 1–4 as one command sheet
teardown.sh	END OF PROGRAMME only

Trainer preparation

Same per-project preparation as the IAM session (prep-project.sh in that kit): APIs, the org-policy override, and the Cloud Build roles on the compute default service account. Add clouddeploy.googleapis.com, containerscanning.googleapis.com and ondemandscanning.googleapis.com — Lab 1 step 1 enables them too. A first build in a fresh project takes a minute longer while APIs warm up.

With a hosted repository (not used today)

Connect the repository to Cloud Build (repositories 2nd gen, GitHub App), create a push trigger on main pointing at cloudbuild.yaml with _SHA=\$SHORT_SHA, and a pull-request trigger that runs the gates without the push/deploy steps; make the Cloud Build check required in branch protection. Everything in this manual then happens on every merge instead of on gcloud builds submit.

What this session leaves behind

- Cloud Deploy pipeline bookshelf-api with staging and prod targets; releases r-<sha>, r-bad (rolled back), r-fixed
- bookshelf-api on 100 % of a good revision, still on bookshelf-api-min, internal ingress, secret mounted
- bookshelf-api-staging (public ingress, authenticated) — the Observability session uses both
- the git history in bookshelf-pipeline/ and the images tagged by commit in Artifact Registry